



Meta-aprendizagem

Avaliando recomendações de sistemas de Meta-aprendizagem / AutoML

Prof. Lucas Amorim - lucas@ic.ufal.br

1

Sumário



- Introdução
- Metodologia para avaliar algoritmos de nível base
- Normalização da performance para algoritmos de nível base
- Metodologia para avaliar sistemas de meta-aprendizagem e AutoML
- Avaliação de recomendações por meio de correlação
- Avaliação dos efeitos da recomendação
- Mais algumas medidas úteis
- Exercício

2

Introdução



- Na aula anterior vimos como criar sistemas de meta-aprendizagem capazes de gerar recomendações de diferentes tipos.
- Fizemos uma avaliação preliminar sobre a qualidade das recomendações geradas.
- Vamos agora aprofundar nosso estudo sobre como avaliar as recomendações tanto em nível de meta-modelo como em nível base.
- Basicamente buscamos responder:
 1. A performance do sistema de meta-aprendizagem/AutoML em questão é satisfatória do ponto de vista do usuário?
 2. Como a performance deste sistema se compara com outros sistemas deste tipo (concorrentes) ?

3

Introdução



- Frequentemente, sistemas de meta-aprendizagem (MtL) ou AutoML recomendam rankings, pelas razões já discutidas.
- Há duas maneiras de avaliar a qualidade da recomendação de rankings de algoritmos:
 - Comparar o ranking recomendado com algum padrão ouro.
 - Avaliar os efeitos em nível base quando essas recomendações são seguidas.

4

Metodologia para avaliação de algoritmos em nível base



- Avaliação de algoritmos é algo discutido em livros/disciplinas de aprendizagem de máquina.
- Alguns conceitos básicos precisam ser brevemente revisados.
- **Erro de generalização:** é o erro que um modelo gerado por um algoritmo comete em instâncias não vistas durante o treinamento.
- Durante o treinamento de um algoritmo em um dataset, não se busca um *fit* perfeito nesses dados, e sim gerar um modelo que obtenha boa performance em pontos de dados ainda não vistos.
- Veremos como o erro de generalização pode ser medido.

5

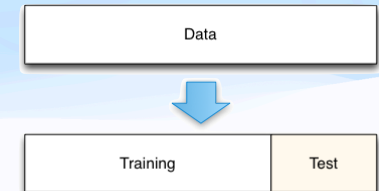
Metodologia para avaliação de algoritmos em nível base



Estratégias de avaliação

Holdout - O conjunto de dados é dividido em duas partes, treinamento (ex.: 90%) e teste (ex.: 10%).

- O algoritmo é treinado com o conjunto de treinamento e sua performance é avaliada no conjunto de teste.
- A proporção da divisão costuma afetar o resultado.



6

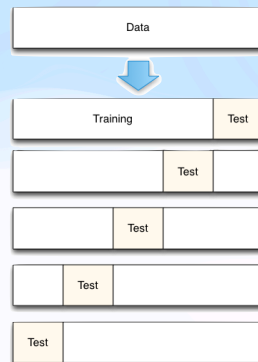
Metodologia para avaliação de algoritmos em nível base



Estratégias de avaliação

Validação cruzada - O conjunto de dados é dividido em k partes (ex.: 5 partes: Validação cruzada 5-fold).

- O algoritmo é treinado com $k - 1$ partes e testado na parte remanescente. Isso se repete k vezes, sempre alternando qual das partes será a porção de teste.
- São obtidos k valores de performances que podem então ser agregados.
- Diminui a variabilidade encontrada no holdout.
- Em problemas de classificação costuma-se usar a **validação cruzada estratificada**.



7

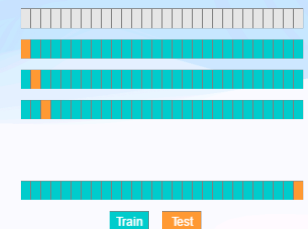
Metodologia para avaliação de algoritmos em nível base



Estratégias de avaliação

Leave one out CV - Uma instância é deixada para teste e as demais para treino.

- Em um conjunto com n instâncias, n avaliações são realizadas, em que é alternada a instância de teste.
- As previsões individuais são agregadas e a performance no conjunto de teste é medida.



8

Metodologia para avaliação de algoritmos em nível base



Função de Perda

Função de Perda (loss) - Em aprendizagem de máquina é comum utilizar o termo perda para se referir ao erro de predição.

- Dado um algoritmo a^j com hiperparâmetros θ e dado um dataset d . A função de perda pode ser definida por:

$$L = \mathcal{L}(a_\theta^j, d_{train}, d_{valid})$$

- Alternativamente, se $M(a_\theta^j, d_{train})$ representar a saída do modelo treinado pelo algoritmo a_θ^j sobre os dados de d_{train} , então:

$$\hat{y}_{valid} = A(M(a_\theta^j, d_{train}), X_{valid})$$

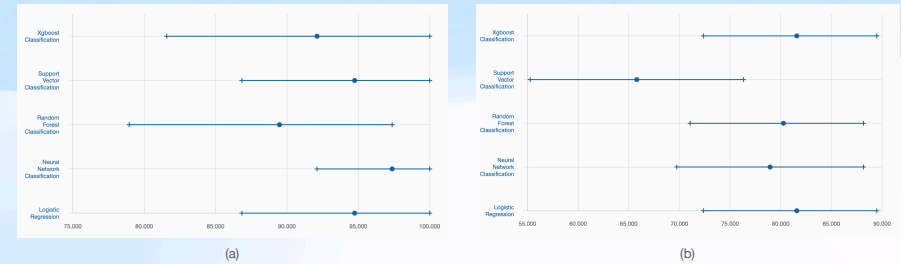
$$L = \mathcal{L}(\hat{y}_{valid}, y_{valid})$$

9

Normalização da performance de algoritmos de nível base



- É comum que o intervalo de variação da performance de classificação para diferentes datasets varie bastante.
- Uma acurácia de 90% pode ser considerada alta para o dataset (b) mas baixa para o dataset (a).



Acurácia de classificação nos datasets (a) Iris e (b) Heart Disease. Fonte: UCI Machine Learning Repository.

<https://archive.ics.uci.edu>

10

Normalização da performance de algoritmos de nível base



- Se quisermos comparar a performance de sistemas em **diferentes** datasets, é importante escalonar os valores das performances.
- Diferentes abordagens podem ser utilizadas:
 - Substituir os valores de performances por ranks
 - Escalonar no intervalo de 0 a 1
 - Mapear em uma distribuição normal (normalizar)
 - Outras formas de escalonar

11

Normalização da performance de algoritmos de nível base

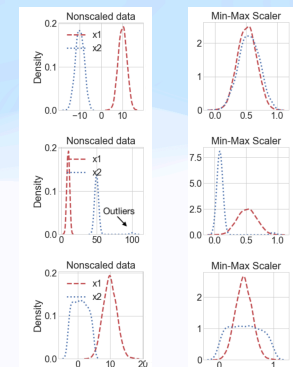


Escalonar no intervalo de 0 a 1

- Para escalonar entre 0 e 1 pode-se usar o escalonador min-max (Min-max Scaler).
- Se o valor original é x_i e o valor mínimo de performances no dataset é x_{min} , então o valor transformado x'_i é:

$$x'_i = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

- Problema: sensível a outliers.



12

Normalização da performance de algoritmos de nível base

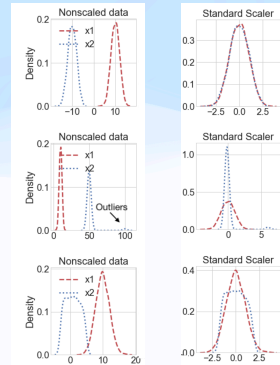


Mapear em uma distribuição normal

- Para mapear em uma distribuição normal, pode-se usar a normalização Z-score (Standard Scaler).

$$x'_i = \frac{x_i - \bar{x}}{s}$$

- Problema: também sensível a *outliers*.



13

Normalização da performance de algoritmos de nível base

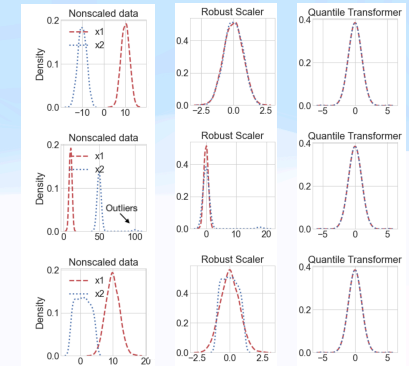


Outras formas de escalonar

- Outras técnicas levam em conta a existência de *outliers*.
- Robust Scaler:** escalona com a distância interquartil.

$$x'_i = \frac{x_i - Q_2(x)}{Q_3(x) - Q_1(x)}$$

- Quantile Transformer:** Transformação não-linear que muda a forma da distribuição e torna os dados mais gaussianos.



14

Normalização da performance de algoritmos de nível base



Outras formas de escalonar

- Embora a literatura tenha proposto maneiras de escalonar as performances, não temos conhecimento de estudos do efeito da escolha da técnica de normalização do meta-alvo (y) na performance do meta-modelo.
- Em Amorim et al. (2022) estudamos o efeito da escolha da técnica de escalonamento para ajuste das variáveis preditivas (X) em tarefas de aprendizagem de máquina.
- Problema interessante em aberto...



DOI: 10.1016/j.asoc.2022.109924
github.com/amorimb/scaling_matters

Lucas B.V. de Amorim, George D.C. Cavalcanti, Rafael M.O. Cruz (2022). The choice of scaling technique matters for classification performance, Applied Soft Computing, Volume 133, 2023.

15

Normalização da performance de algoritmos de nível base



Outras formas de escalonar

- Outra ideia foi proposta por Gama e Brazdil (1995).
- A proposta é que valores sejam expressos em termos da margem de erro (EM).
 - Primeiro obtém-se as taxas de erro (ER) de cada algoritmo no dataset (1 - acurácia).
 - Em seguida calcula-se a margem de erro: $EM = \sqrt{\frac{ER \times (1 - ER)}{NT}}$, onde NT é o número de instâncias do conjunto de teste.
 - Agora, a taxa de erro de cada algoritmo é convertida em valores indicando quantos EMs ele está abaixo do melhor algoritmo.

Gama, J. and Brazdil, P. (1995). Characterization of classification algorithms. In Pinto Ferreira, C. and Mamede, N. J., editors, Progress in Artificial Intelligence, Proceedings of the Seventh Portuguese Conference on Artificial Intelligence, pages 189–200. Springer Verlag.

16

Metodologia para avaliar sistemas de meta-aprendizagem e AutoML



- Para avaliar a qualidade de sistemas de meta-aprendizagem e AutoML podemos seguir uma dessas duas abordagens:
- 1. Avaliação em etapa única com hold-out (one-pass with hold-out)
- 2. Avaliação com validação cruzada

17

Metodologia para avaliar sistemas de meta-aprendizagem e AutoML



Avaliação em etapa única com hold-out

- A primeira forma da avaliação baseia-se na divisão do dataset em um conjunto de treinamento e um conjunto de teste.
- Ao comparar a performance de vários sistemas de meta-aprendizagem/AutoML, esta divisão precisa ser constante em todas as comparações.
- Muitos sistemas realizam esta avaliação internamente para determinar qual algoritmo base deve ser recomendado.
- O objetivo de sistemas de meta-aprendizagem/AutoML pode ser formalizado como uma busca pela minimização de uma função de perda. Por exemplo, em problemas do tipo CASH, Thornton et al. (2013) define que buscamos pelo algoritmo a_{θ}^* tal que:

$$a_{\theta}^* = \arg \min_{a' \in A, \theta \in \Theta^j} \mathcal{L}(a_{\theta}^j, d_{\text{train}}, d_{\text{valid}})$$

18

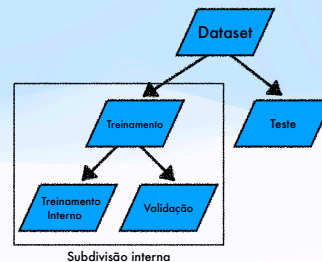
Metodologia para avaliar sistemas de meta-aprendizagem e AutoML



Avaliação em etapa única com hold-out

Avaliação interna realizada por sistemas de meta-aprendizagem/AutoML

- Com o intuito de melhorar a qualidade do sistema de recomendação gerado, muitos sistemas realizam um processo interno com o objetivo de ajustar parâmetros ou comparar variantes em busca da melhor.
- Para tanto os dados de treinamento são novamente divididos em *treinamento interno* e *validação*.
- A variante com melhor performance no conjunto de validação é selecionada para compor o sistema.
- Possivelmente é retreinada em todo o conjunto de treinamento.
- Finalmente, o modelo é avaliado nos dados de teste.
- A avaliação interna pode incluir validação-cruzada para maior estabilidade (menor dispersão). Desvantagem: maior custo/tempo.
- Note que no meta-nível ainda estamos utilizando *hold-out*.



19

Metodologia para avaliar sistemas de meta-aprendizagem e AutoML



Avaliação em etapa única com hold-out

Evitar avaliação enviesada

- Ao avaliar um sistema de recomendação ou AutoML que utilize meta-aprendizagem é importante garantir que não haja vazamento de dados.
- Ou seja: que um dataset presente no meta-dataset usado para treinamento do sistema não apareça no conjunto de usado para avaliar o sistema.
- Isso levaria a uma estimativa enviesada de performance já que o sistema tende a funcionar melhor em um dataset conhecido. (lembre-se: no meta-nível, dataset = instância)
- Alguns sistemas, como Auto-sklearn, são treinados em um meta-dataset contendo muitos datasets comuns em benchmarks. Portanto, ao comparar esse sistema com outro, os datasets usados para avaliar precisam ser diferentes dos que estão no meta-dataset.

20

Metodologia para avaliar sistemas de meta-aprendizagem e AutoML



Avaliação com validação cruzada

- A outra forma de avaliação consiste em realizar múltiplas etapas de treino/teste em um esquema de validação cruzada.
- Os vários datasets (instâncias) do meta-dataset podem ser divididos em folds. Para cada fold a avaliação nos datasets de teste resulta em um valor de performance.
- Os valores de performance são então agregados para determinar a performance do sistema com mais baixa dispersão e menor influência da "sorte" do que na técnica hold-out.
- Em muitos domínios o meta-dataset pode ser relativamente pequeno e então é viável, e recomendável, o emprego da validação cruzada leave-one-out (LOOCV). Às vezes referida por leave-one-dataset-out nesse contexto.

21

Metodologia para avaliar sistemas de meta-aprendizagem e AutoML



Avaliação com validação cruzada

Avaliação do meta-modelo com consultas a tabela

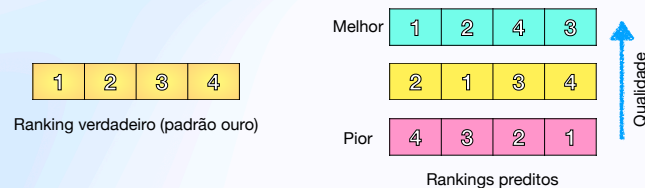
- Como a avaliação dos algoritmos de nível base pode ser computacionalmente custosa, é comum utilizar a seguinte estratégia:
- A performance de cada algoritmo em cada dataset é avaliada uma vez e registrada em uma tabela (que afinal comporá o meta-dataset).
- Sempre que necessário conduzir a mesma avaliação novamente, ao invés de reexecutar o modelo base, os resultados são buscados na tabela.

22

Avaliação de recomendações por meio de correlação



- Como vimos, muitos sistemas de MTL/AutoML recorrem à recomendação de rankings.
- A qualidade desses rankings é tipicamente estabelecida por meio de comparação com um **padrão ouro**, ou seja, o ranking ideal no dataset de teste.
- Às vezes o ranking ideal é referido como ranking verdadeiro (*true ranking*).
- Diferentes rankings preditos têm diferentes graus de qualidade, por exemplo:



23

Avaliação de recomendações por meio de correlação



Medidas de correlação entre rankings

- Várias medidas podem ser empregadas para medir a correlação do ranking predito com o ranking verdadeiro:
- Spearman's rank Correlation (SRC)
- Kendall Tau Correlation
- Weighted Tau Correlation
- Weighted Rank Correlation
- Vejamos a definição de cada uma dessas técnicas e um exemplo de aferição de rankings com as mesmas.

24

Avaliação de recomendações por meio de correlação

Spearman's Rank Correlation (SRC)

$$SRC(\hat{R}, R) = \frac{\sum_{i=1}^n (\hat{R}_i - \bar{\hat{R}})(R_i - \bar{R})}{\sqrt{\sum_{i=1}^n (\hat{R}_i - \bar{\hat{R}})^2 \sum_{i=1}^n (R_i - \bar{R})^2}}$$

Onde $\hat{R}_i$ e R_i representam os ranks predito e verdadeiro para o item i e n o número de itens.

- É similar a um cálculo de regressão (razão entre covariância e variância).
- Mede a monotonicidade entre duas variáveis. i.e. crescem juntas ou decrescem juntas.

25

Avaliação de recomendações por meio de correlação

Coeficiente de correlação Tau de Kendall

$$\tau = \frac{(\text{quantidade de pares concordantes}) - (\text{quantidade de pares discordantes})}{n(n-1)/2}$$

26

Avaliação de recomendações por meio de correlação

Medida ponderada de correlação de rank (Weighted rank measure of correlation)

$$r_w(\hat{R}, R) = 1 - \frac{6 \sum_{i=1}^n (\hat{R}_i - R_i)^2 ((n - \hat{R}_i + 1) + (n - R_i + 1))}{n^4 + n^3 - n^2 - n}$$

- A ideia aqui é que a medida seja mais afetada por itens com ranks mais baixos do que outros com ranks mais altos.
- Note que temos um termo no denominador que representa o peso: $w = ((n - \hat{R}_i + 1) + (n - R_i + 1))$.
- Exemplo para um caso com $n = 10$.
- Erro no 1º lugar: Se a técnica era 1ª e foi prevista como 2ª: O peso seria $(10 - 2 + 1) + (10 - 1 + 1) = 9 + 10 = 19$.
- Erro no 9º lugar: Se a técnica era 9ª e foi prevista como 10ª: O peso seria $(10 - 10 + 1) + (10 - 9 + 1) = 1 + 2 = 3$.

27

Avaliação de recomendações por meio de correlação

Exemplo

- Exemplos de rankings e valores das medidas:

A	1	2	3	4	5	6
B	1	2	3	4	5	6
C	2	1	3	4	5	6
D	1	2	3	4	6	5
E	6	5	4	3	2	1

SRC(A, B) = 1.00	τ (A, B) = 1.00	τ_w (A, B) = 1.00	r_w (A, B) = 1.00
SRC(A, C) = 0.94	τ (A, C) = 0.87	τ_w (A, C) = 0.94	r_w (A, C) = 0.91
SRC(A, D) = 0.94	τ (A, D) = 0.87	τ_w (A, D) = 0.76	r_w (A, D) = 0.98
SRC(A, E) = -1.00	τ (A, E) = -1.00	τ_w (A, E) = -1.00	r_w (A, E) = -1.00

Obs.: Implementação em Python no repositório da disciplina.

28

Avaliação dos efeitos das recomendações



- As medidas de correlações sejam úteis para comparar o ranking recomendado com um padrão ouro, ou seja a **performance de meta-modelo**.
- No entanto, é importante avaliar o efeito das recomendações, ou seja, qual é a melhoria de performance em **nível base** ao seguir as recomendações.
- Muitos pesquisadores adotaram uma abordagem que simula uma avaliação sequencial dos algoritmos (conforme ranking recomendado) no dataset de teste.
- Para visualizar os resultados, usa-se comumente a curva de *perda de performance*.

29

Avaliação dos efeitos das recomendações



Curvas de perda de performance

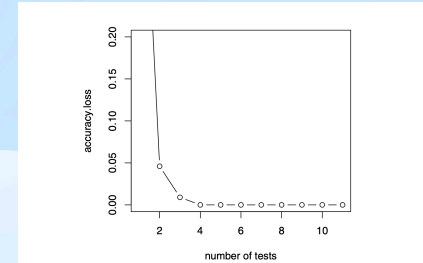


Fig. 3.2: Loss curves depending on the number of tests

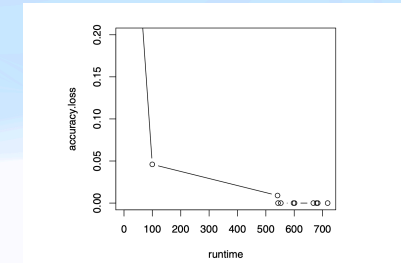


Fig. 3.3: Loss curves depending on runtime

30

Avaliação dos efeitos das recomendações



Agregando curvas após LOOCV

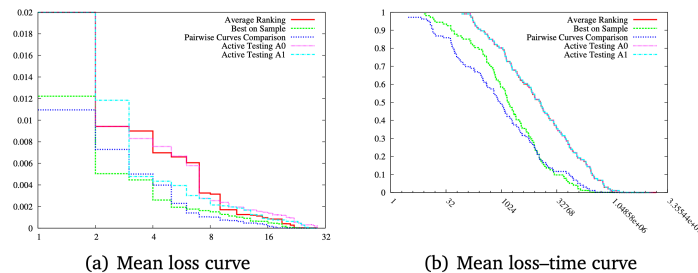


Fig. 3.4: Mean loss and loss-time curves of five metalearning systems obtained on 105 datasets. Image taken from van Rijn (2016)

van Rijn, J. N. (2016). Massively collaborative machine learning. PhD thesis, Leiden University.

31

Avaliação dos efeitos das recomendações



Outras formas de avaliar os efeitos das recomendações

1. Performance do primeiro lugar: Tomar a performance do primeiro lugar como a performance de nível base da recomendação.
 - Vantagens: Simples de implementar, baixo custo computacional, e fácil de comparar com sistemas que recomendam um único algoritmo.
 - Desvantagem: Não aproveita o potencial do ranking como forma de recomendação.
2. Best of n (ou n -shot): Testar os n primeiros algoritmos do ranking no dataset de teste, tomar a performance do melhor como a performance de nível base da recomendação.
 1. Vantagens: A chance de um bom algoritmo estar entre os n primeiros cresce com n . Ideal para situações com orçamento de tempo.
 2. Desvantagem: Computacionalmente mais caro, mas faz sentido em situações com muitos algoritmos (ou muitos hiperparâmetros em um problema CASH).

32

Fig. 9 Performance distributions of the pipelines recommended by each AutoML method on the tested datasets

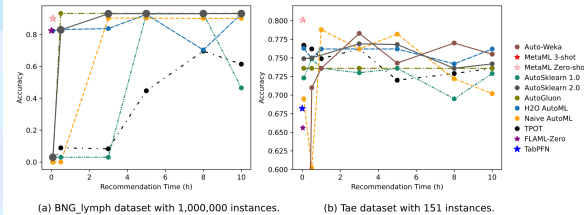
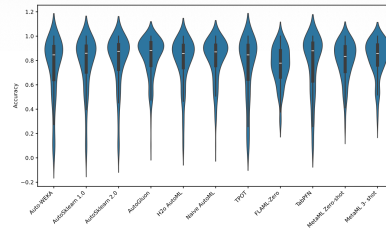


Fig. 10 Comparison of AutoML methods using different recommendation time budgets

CM Maia, LBV de Amorim, GDC Cavalcanti, RMO Cruz (2025). MetaML: a multi-label meta-learning approach for pipeline recommendation. Machine Learning 114

33

Avaliação dos efeitos das recomendações

Testes estatísticos e diagrama de CD

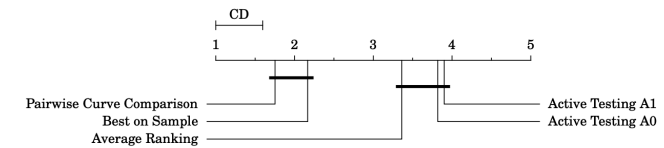


Fig. 3.5: A typical result obtained with the Nemenyi test. Image taken from van Rijn (2016)

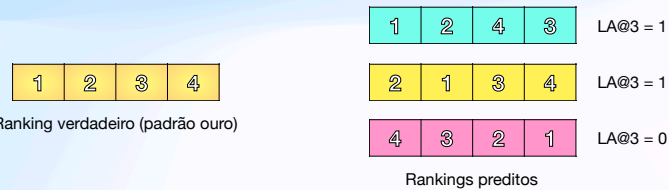
van Rijn, J. N. (2016). Massively collaborative machine learning. PhD thesis, Leiden University.

34

Mais algumas medidas úteis

Loose accuracy (LA@X)

- Esta medida usa a acurácia do ranking dos top X elementos na lista rankeada (Kalousis, 2002).
- LA@X retorna o valor 1 se o entre os X top elementos estiver incluído o melhor no ranking verdadeiro, senão, retorna 0.



Kalousis, A. (2002). Algorithm Selection via Meta-Learning. PhD thesis, University of Geneva, Department of Computer Science.

35

Mais algumas medidas úteis

Normalized discounted cumulative gain (NDCG)

- O Discounted cumulative gain (DCG), e sua forma normalizada (NDCG), são usados para avaliar a efetividade de motores de busca e têm sido também adotados em alguns trabalhos de seleção de algoritmos.
- Esta função usa uma escala de relevância graduada de acordo com a posição correspondente na lista rankeada.
- CG é a soma simples da relevância de todos os itens recomendados.
- Ex.: se um sistema recomenda 3 técnicas: Técnica 1 (relevância 3), Técnica 2 (relevância 2), Técnica 3 (relevância 0). $CG = 3 + 2 + 0 = 5$
- DCG aplica um desconto de acordo com a posição, de forma a penalizar rankings em que a técnicas mais relevantes não estão na frente:

$$DCG_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$$

- O NDCG simplesmente normaliza esse valor, dividindo-o pelo IDCG, que é o DCG ideal, para que o resultado fique entre 0 e 1: $NDCG = DCG/IDCG$.
- Um NDCG = 1 significa ranking perfeito.
- Em um sistema de recomendação de algoritmos, podemos usar a performance do algoritmo como a relevância para o cálculo do NDCG.

36

Exercício



Partindo do seu código da Tarefa 3, realize as seguintes mudanças:

A) Crie um novo conjunto de datasets de treinamento. Utilize a API do OpenML para baixar entre 40 e 200 datasets segundo seus próprios critérios (dica: evite datasets com "área" (n_linhas x n_colunas) muito grande).

B) Além das duas abordagens de ranking já implementadas na Tarefa 3, implemente também uma abordagem simples de Average Ranking, para servir de baseline. Por tanto, ficaremos com essas 3 abordagens para comparar:

1. Average Ranking: Simplesmente calcule o ranking médio de cada classificador no meta-dataset de treino e use este ranking médio para a fase de recomendação.
2. (Aproveitando da Tarefa 3) Predizer performances, converter em rankings : Manter a matriz de performances original como meta-alvo, treinar o meta-modelo como um regressor de múltiplas saídas para predizer as performances, em seguida, converter as performances preditas em rankings.
3. (Aproveitando da Tarefa 3) Converter performances em rankings, predizer rankings: Converter a matriz de performances em uma matriz de rankings e utilizar os rankings como meta alvo. Treinar o meta-modelo como um regressor de múltiplas saídas para predizer diretamente o ranking.

Avalie as três abordagens com LOOCV e ...

C) Compare-as em nível base utilizando box plots das performances do 1º colocado dos rankings recomendados.

D) Compare-as em nível base utilizando curvas de perda (Fig. 3.4(a) do livro).

D) Compare as três abordagens em nível meta de acordo com as métricas SRC, r_w , NDCG e LA@3.

Obs.: Gere tabelas e gráficos para representar seus resultados. Crie, no notebook, células textuais apresentando e discutindo esses resultados. Envie na tarefa o link para o Google Colab.

<https://github.com/amorimlb/Meta-aprendizagem-2026.1-code>